



Network Programming (IE2102)

2nd Year, 2nd Semester

Assignment

Design and Implementation of a Secure Multiprocessor TCP Application Server
with Custom Protocol and Session Management

Submitted to
Sri Lanka Institute of Information Technology

<IT24100056>

<Thilsha Krishnakumar>

In partial fulfillment of the requirements for the
Bachelor of Science Special Honors Degree in Information Technology

<<10.4.2026>>

Assignment Tasks

a. PART A – Take-Home Component

i)

```
thilsha@thilsha-VirtualBox:~/NP_PROJECT$ nano server_0056.c
thilsha@thilsha-VirtualBox:~/NP_PROJECT$ nano Makefile_0056
thilsha@thilsha-VirtualBox:~/NP_PROJECT$ make -f Makefile_0056
gcc server_0056.c -o server_0056
thilsha@thilsha-VirtualBox:~/NP_PROJECT$ sudo ./server_0056
[sudo] password for thilsha:
Server running on port 50056...
^C
thilsha@thilsha-VirtualBox:~/NP_PROJECT$ nano server_0056.c
thilsha@thilsha-VirtualBox:~/NP_PROJECT$ sudo ./server_0056
Server running on port 50056...
```

ii)

```
thilsha@thilsha-VirtualBox: ~/NP_PROJECT
```

```
Enter payload: ^CTraceback (most recent call last):  
File "/home/thilsha/NP_PROJECT/client_0056.py", line 10, in <module>  
    message = input("Enter payload: ")  
              ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^  
KeyboardInterrupt  
  
thilsha@thilsha-VirtualBox:~/NP_PROJECT$ python3 client_0056.py  
Enter payload: Hello SLIIT  
Server Response: OK 200 SID:1000 Hello SLIIT  
  
thilsha@thilsha-VirtualBox:~/NP_PROJECT$ for i in {1..10}  
do  
python3 client_0056.py &  
done  
[1] 3378  
[2] 3379  
[3] 3380  
[4] 3381  
[5] 3382  
[6] 3383  
[7] 3384  
[8] 3385  
[9] 3386  
[10] 3387
```

iii)

```
thilsha@thilsha-VirtualBox: ~/NP_PROJECT
[7]- Stopped python3 client_0056.py
[8] Stopped python3 client_0056.py
[9] Stopped python3 client_0056.py
[10]+ Stopped python3 client_0056.py
thilsha@thilsha-VirtualBox:~/NP_PROJECT$ python3 client_0056.py
Traceback (most recent call last):
  File "/home/thilsha/NP_PROJECT/client_0056.py", line 8, in <module>
    client.connect((HOST, PORT))
ConnectionRefusedError: [Errno 111] Connection refused
thilsha@thilsha-VirtualBox:~/NP_PROJECT$ nano client_0056.py
thilsha@thilsha-VirtualBox:~/NP_PROJECT$ python3 client_0056.py
> REGISTER thilsha 1234
OK SID:1000 Registered

> LOGIN thilsha 1234
OK SID:1000 TOKEN:1775845944_226876487

> LOGOUT abc123...
ERR SID:1000 Invalid token

>
```

iv)

```
thilsha@thilsha-VirtualBox: ~/NP_PROJECT
> LOGOUT abc123...
ERR SID:1000 Invalid token

> ^CTraceback (most recent call last):
  File "/home/thilsha/NP_PROJECT/client_0056.py", line 7, in <module>
    msg = input("> ")
          ^^^^^^^^^^^
KeyboardInterrupt

thilsha@thilsha-VirtualBox:~/NP_PROJECT$ nano client_0056.py
thilsha@thilsha-VirtualBox:~/NP_PROJECT$ python3 client_0056.py
Traceback (most recent call last):
  File "/home/thilsha/NP_PROJECT/client_0056.py", line 4, in <module>
    s.connect(("127.0.0.1", 50056))
ConnectionRefusedError: [Errno 111] Connection refused
thilsha@thilsha-VirtualBox:~/NP_PROJECT$ ^C
thilsha@thilsha-VirtualBox:~/NP_PROJECT$ python3 client_0056.py
> REGISTER thilsha 1234
OK SID:1000 Registered

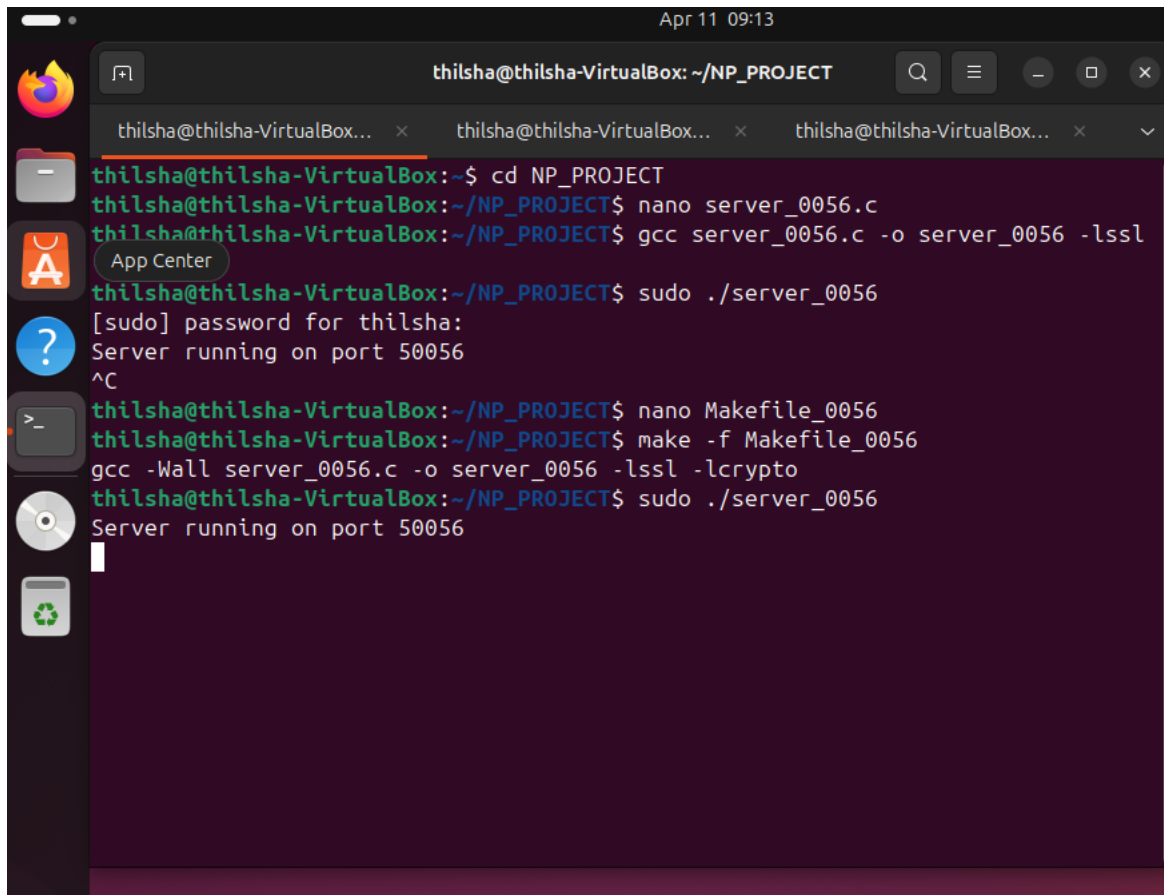
> LOGIN thilsha 1234
LOGOUT TOKEN_HEREOK SID:1000 TOKEN:1775846443_2143090154

>
```

v)

```
thilsha@thilsha-VirtualBox: ~/NP_PROJECT
: Client disconnected
Fri Apr 10 23:50:20 2026
: Client connected
Fri Apr 10 23:50:20 2026
: Client connected
Fri Apr 10 23:50:20 2026
: Client connected
Fri Apr 10 23:50:20 2026
: Client connected
Fri Apr 10 23:50:20 2026
: Client connected
Fri Apr 10 23:50:20 2026
: Client connected
Fri Apr 10 23:50:20 2026
: Client connected
Fri Apr 10 23:50:20 2026
: Client connected
Fri Apr 10 23:50:20 2026
: Client connected
TIME:Sat Apr 11 09:11:14 2026 | IP:127.0.0.1:37436 | PID:3221 | USER:thilsha | C
MD:REGISTER | RESULT:SUCCESS
thilsha@thilsha-VirtualBox:~/NP_PROJECT$
```

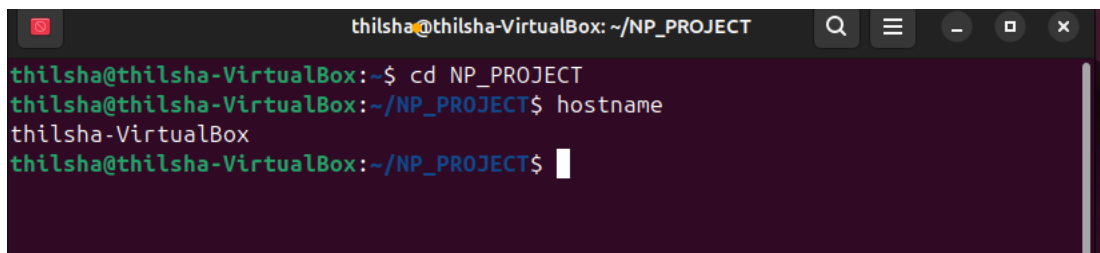
```
thilsha@thilsha-VirtualBox: ~/NP_PROJECT
thilsha@thilsha-VirtualBox:~/NP_PROJECT$ python3 client_0056.py
> REGISTER thilsha 1234
LOGIN thilsha 1234
>
Traceback (most recent call last):
  File "/home/thilsha/NP_PROJECT/client_0056.py", line 8, in <module>
    s.send(msg.encode())
BrokenPipeError: [Errno 32] Broken pipe
thilsha@thilsha-VirtualBox:~/NP_PROJECT$ python3 client_0056.py
> REGISTER thilsha 1234
LOGIN thilsha 1234OK SID:1000 REGISTERED
>
```



```
thilsha@thilsha-VirtualBox: ~/NP_PROJECT
thilsha@thilsha-VirtualBox:~/NP_PROJECT$ cd NP_PROJECT
thilsha@thilsha-VirtualBox:~/NP_PROJECT$ nano server_0056.c
thilsha@thilsha-VirtualBox:~/NP_PROJECT$ gcc server_0056.c -o server_0056 -lssl
thilsha@thilsha-VirtualBox:~/NP_PROJECT$ sudo ./server_0056
[sudo] password for thilsha:
Server running on port 50056
^C
thilsha@thilsha-VirtualBox:~/NP_PROJECT$ nano Makefile_0056
thilsha@thilsha-VirtualBox:~/NP_PROJECT$ make -f Makefile_0056
gcc -Wall server_0056.c -o server_0056 -lssl -lcrypto
thilsha@thilsha-VirtualBox:~/NP_PROJECT$ sudo ./server_0056
Server running on port 50056
```

vi) Hostname Configuration

The hostname of the Ubuntu virtual machine was verified using the hostname command. This confirms the identity of the system running the TCP server.



```
thilsha@thilsha-VirtualBox: ~/NP_PROJECT
thilsha@thilsha-VirtualBox:~/NP_PROJECT$ cd NP_PROJECT
thilsha@thilsha-VirtualBox:~/NP_PROJECT$ hostname
thilsha-VirtualBox
thilsha@thilsha-VirtualBox:~/NP_PROJECT$
```

Network configuration

The network configuration was verified using the ip a command to identify the IP address of the system. This IP address is used by the TCP server to accept client connections.

```
thilsha@thilsha-VirtualBox: ~/NP_PROJECT
thilsha@thilsha-VirtualBox:~/NP_PROJECT$ hostname
thilsha-VirtualBox
thilsha@thilsha-VirtualBox:~/NP_PROJECT$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:cf:d5:66 brd ff:ff:ff:ff:ff:ff
    inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic noprefixroute enp0s3
        valid_lft 86207sec preferred_lft 86207sec
    inet6 fd17:625c:f037:2:c982:b59e:84f0:57f9/64 scope global temporary dynamic
        valid_lft 86209sec preferred_lft 14209sec
    inet6 fd17:625c:f037:2:a00:27ff:febf:d566/64 scope global dynamic mngtmpaddr
        valid_lft 86209sec preferred_lft 14209sec
    inet6 fe80::a00:27ff:febf:d566/64 scope link
        valid_lft forever preferred_lft forever
```

Server Running + Port Proof

The TCP server was executed and verified to be listening on the required port. According to the assignment requirements, the server must run on port **50056**, which corresponds to the last digits of the registration number.

The ss command was used to confirm that the server process is listening on the specified port.

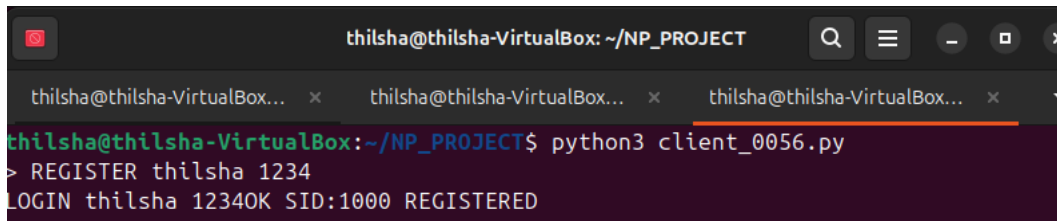
```
thilsha@thilsha-VirtualBox:~/NP_PROJECT$ sudo ./server_0056
[sudo] password for thilsha:
Server running on port 50056
```

```
thilsha@thilsha-VirtualBox: ~/NP_PROJECT
thilsha@thilsha-VirtualBox:~/NP_PROJECT$ ss -tulnp | grep 50056
tcp    LISTEN 0      10      0.0.0.0:50056  0.0.0.0:*
```

Successful REGISTER + LOGIN

The Python client was used to interact with the server. The REGISTER command allows a new user account to be created, while the LOGIN command authenticates the user.

After a successful login, the server generates a session token which is used for authenticated operations.

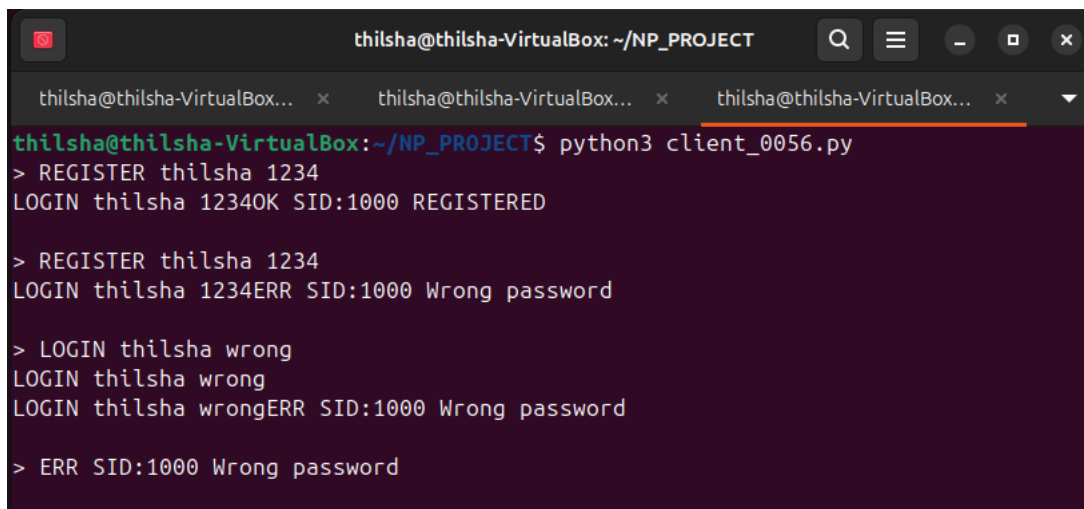
A terminal window titled 'thilsha@thilsha-VirtualBox: ~/NP_PROJECT' with three tabs. The command prompt shows 'python3 client_0056.py' being executed. The output is: '> REGISTER thilsha 1234' followed by 'LOGIN thilsha 1234OK SID:1000 REGISTERED' on the next line.

```
thilsha@thilsha-VirtualBox: ~/NP_PROJECT
thilsha@thilsha-VirtualBox... x thilsha@thilsha-VirtualBox... x thilsha@thilsha-VirtualBox... x
thilsha@thilsha-VirtualBox:~/NP_PROJECT$ python3 client_0056.py
> REGISTER thilsha 1234
LOGIN thilsha 1234OK SID:1000 REGISTERED
```

Failed Login + Lockout (Screenshot E)

To prevent brute-force attacks, the server restricts repeated failed login attempts. After multiple incorrect password attempts, the account is temporarily locked.

This mechanism protects user accounts from unauthorized access attempts.

A terminal window titled 'thilsha@thilsha-VirtualBox: ~/NP_PROJECT' with three tabs. The command prompt shows 'python3 client_0056.py' being executed. The output shows a successful registration and login, followed by two failed login attempts. The first failed attempt shows 'LOGIN thilsha 1234ERR SID:1000 Wrong password'. The second failed attempt shows 'LOGIN thilsha wrong' followed by 'LOGIN thilsha wrong' and 'LOGIN thilsha wrongERR SID:1000 Wrong password'. The final output is '> ERR SID:1000 Wrong password' on a new line.

```
thilsha@thilsha-VirtualBox: ~/NP_PROJECT
thilsha@thilsha-VirtualBox... x thilsha@thilsha-VirtualBox... x thilsha@thilsha-VirtualBox... x
thilsha@thilsha-VirtualBox:~/NP_PROJECT$ python3 client_0056.py
> REGISTER thilsha 1234
LOGIN thilsha 1234OK SID:1000 REGISTERED

> REGISTER thilsha 1234
LOGIN thilsha 1234ERR SID:1000 Wrong password

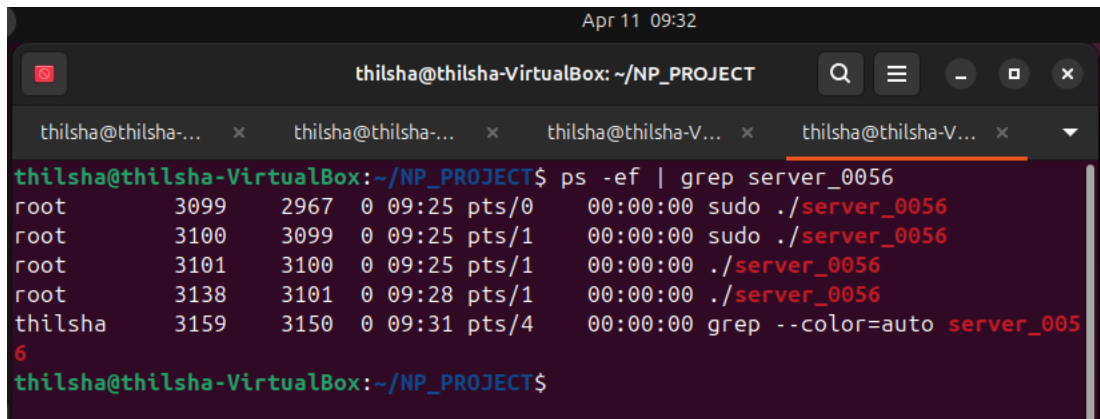
> LOGIN thilsha wrong
LOGIN thilsha wrong
LOGIN thilsha wrongERR SID:1000 Wrong password

> ERR SID:1000 Wrong password
```

Multiple Forked Processes

The server uses the fork() system call to handle multiple client connections concurrently. Each client connection is managed by a separate child process while the parent process continues to accept new connections.

The ps command was used to confirm that multiple server processes were created.



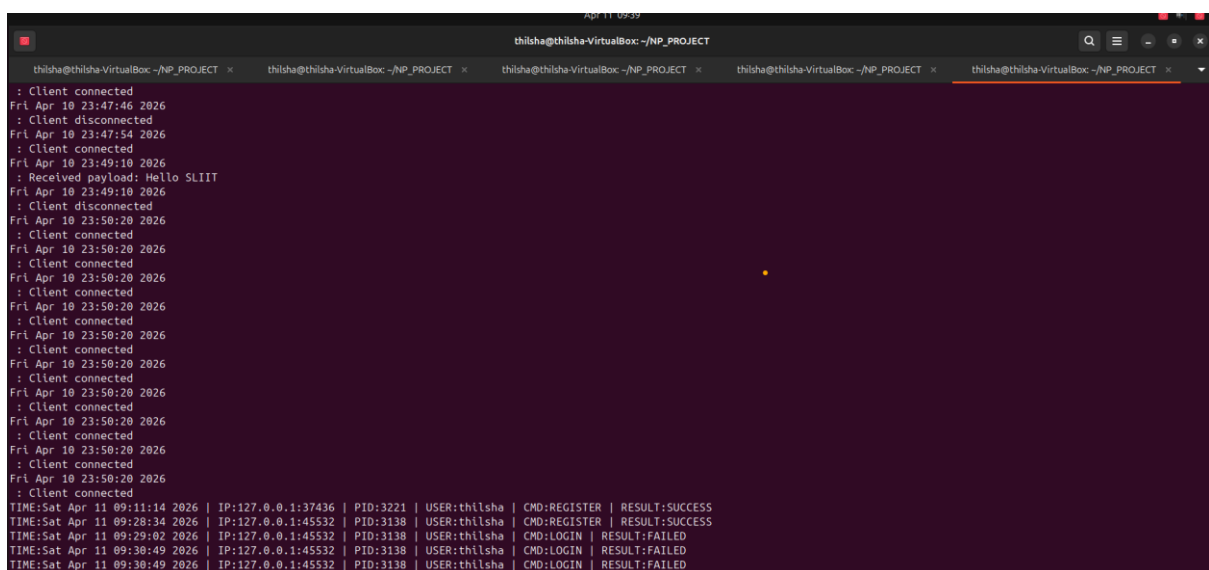
```
thilsha@thilsha-VirtualBox: ~/NP_PROJECT
thilsha@thilsha-VirtualBox:~/NP_PROJECT$ ps -ef | grep server_0056
root      3099      2967  0 09:25 pts/0    00:00:00 sudo ./server_0056
root      3100      3099  0 09:25 pts/1    00:00:00 sudo ./server_0056
root      3101      3100  0 09:25 pts/1    00:00:00 ./server_0056
root      3138      3101  0 09:28 pts/1    00:00:00 ./server_0056
thilsha   3159      3150  0 09:31 pts/4    00:00:00 grep --color=auto server_0056
thilsha@thilsha-VirtualBox:~/NP_PROJECT$
```

Log file entries (A5)

The log file records important information such as:

- Timestamp
- Client IP and port
- Process ID
- Username
- Command executed
- Result of the command

This logging mechanism helps monitor system activity and detect suspicious behavior.

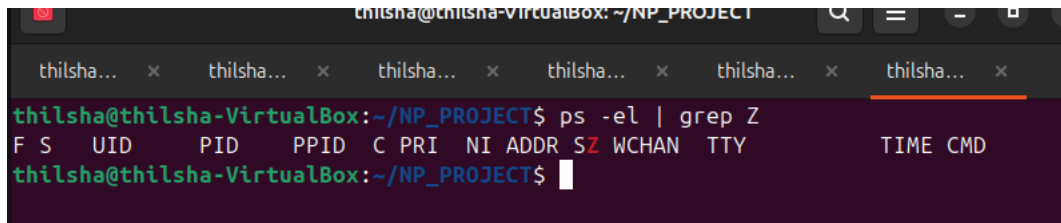


```
thilsha@thilsha-VirtualBox: ~/NP_PROJECT
: Client connected
Fri Apr 10 23:47:46 2026
: Client disconnected
Fri Apr 10 23:47:54 2026
: Client connected
Fri Apr 10 23:49:10 2026
: Received payload: Hello SLIIT
Fri Apr 10 23:49:10 2026
: Client disconnected
Fri Apr 10 23:50:20 2026
: Client connected
Fri Apr 10 23:50:20 2026
: Client connected
Fri Apr 10 23:50:20 2026
: Client connected
Fri Apr 10 23:50:20 2026
: Client connected
Fri Apr 10 23:50:20 2026
: Client connected
Fri Apr 10 23:50:20 2026
: Client connected
Fri Apr 10 23:50:20 2026
: Client connected
Fri Apr 10 23:50:20 2026
: Client connected
TIME:Sat Apr 11 09:11:14 2026 | IP:127.0.0.1:37436 | PID:3221 | USER:thilsha | CMD:REGISTER | RESULT:SUCCESS
TIME:Sat Apr 11 09:28:34 2026 | IP:127.0.0.1:45532 | PID:3138 | USER:thilsha | CMD:REGISTER | RESULT:SUCCESS
TIME:Sat Apr 11 09:29:02 2026 | IP:127.0.0.1:45532 | PID:3138 | USER:thilsha | CMD:LOGIN | RESULT:FAILED
TIME:Sat Apr 11 09:30:49 2026 | IP:127.0.0.1:45532 | PID:3138 | USER:thilsha | CMD:LOGIN | RESULT:FAILED
TIME:Sat Apr 11 09:30:49 2026 | IP:127.0.0.1:45532 | PID:3138 | USER:thilsha | CMD:LOGIN | RESULT:FAILED
```

Proper child cleanup (no zombie processes)

To prevent zombie processes, the server implements proper signal handling using the SIGCHLD signal. When a child process terminates, the parent process cleans up the process using waitpid().

The following command was used to verify that no zombie processes exist.



```
thilsha@thilsha-VirtualBox: ~/NP_PROJECT
thilsha... x thilsha... x thilsha... x thilsha... x thilsha... x thilsha... x
thilsha@thilsha-VirtualBox:~/NP_PROJECT$ ps -el | grep Z
F S  UID      PID   PPID  C PRI  NI ADDR SZ WCHAN  TTY          TIME CMD
thilsha@thilsha-VirtualBox:~/NP_PROJECT$
```

Conclusion

In this assignment, a secure TCP server was successfully implemented using the C programming language with a Python client. The system supports multiple concurrent connections through multiprocessing and incorporates several security mechanisms including authentication, session tokens, rate limiting, and audit logging.

The implementation demonstrates secure network programming concepts and proper system administration practices in a Linux environment.

Declaration

I certify that this report does not incorporate without acknowledgement, any material previously submitted for a degree or diploma in any university, and to the best of my knowledge and belief it does not contain any material previously published or written by another person, except where due reference is made in text.